# SIMATS ENGINEERING
## ASSESSMENT TOOL 2

**COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY**
**COURSE CODE: CSA5116**

---

**COURSE OUTCOME COVERED**

**CO2:** Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4, BL5)

Assessment Tool Weightage: 20%

**QUESTIONS:** **Total Marks: 50**

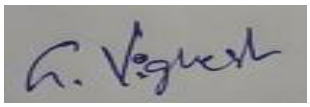
**Annexure A**

## Scenario-Based

---

***Code of Conduct:***

*I Gullapelly Vignesh certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*



***Signature of the student:***

## Annexure A
## Scenario-based

**1. Secure Data Transmission using Block Cipher**
A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext.

A. Explain why this issue occurs.
B. Suggest a more secure block cipher mode and justify your choice based on the scenario.

**2. Choosing an Encryption Algorithm for Banking System**
A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES, and Blowfish.

A. Identify the most suitable algorithm for this scenario.
B. Justify your choice based on security strength and efficiency.

**3. Key Exchange in an Unsecured Network**
Two users want to establish a secure communication channel over a public network using Diffie-Hellman key exchange. However, the network is vulnerable to attacks.

A. Explain how the key exchange works in this scenario.
B. Identify a possible security threat and suggest a solution to mitigate it.

**4. Public Key Cryptography for Email Security**
An organization plans to implement RSA for secure email communication.

A. Describe how RSA can be used for encryption and decryption in this scenario.
B. Explain how key distribution and management play a role in maintaining security.

**5. Cryptography for IoT Devices**
A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

A. Analyze the constraints of IoT devices.
B. Recommend a suitable cryptographic approach and justify your answer.

**RUBRICS FOR CALCULATION:**

| Criteria | Weightage (%) | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Understanding of Scenario | 20% | Demonstrates clear and complete understanding of the scenario and context | Shows good understanding with minor gaps | Partial understanding; misses key aspects | Misinterprets or fails to understand the scenario |
| Identification of Key Issues | 20% | Accurately identifies all relevant issues and factors involved | Identifies most key issues with minor omissions | Identifies limited issues; some irrelevant points | Unable to identify key issues |
| Application of Concepts | 25% | Applies appropriate concepts effectively to address the scenario | Applies concepts with minor errors | Limited or partially correct application | Unable to apply relevant concepts |
| Decision Making | 20% | Makes well-reasoned, logical decisions with strong rationale | Makes reasonable decisions with some justification | Makes weak decisions with limited justification | No clear decisions made or rationale provided |
| Clarity of Response | 15% | Response is clear, structured, and logically presented | Generally clear with minor issues | Somewhat unclear or poorly structured | Disorganized and difficult to understand |

### 3. Key Exchange in an Unsecured Network

**(a) Explain how the key exchange works in this scenario.**

Diffie-Hellman (DH) lets two users, Alice and Bob, agree on a shared secret key over a public network without ever sending the key itself. Its security rests on the discrete logarithm problem: computing g^a mod p is easy, but recovering "a" from the result is computationally infeasible for large numbers.

**Step 1 – Public parameters:** Alice and Bob agree on a large prime number p and a generator g (a primitive root of p). These values are public.

**Step 2 – Private keys:** Alice picks a secret number a and Bob picks a secret number b. These are never transmitted.

**Step 3 – Public values:** Alice computes A = g^a mod p and Bob computes B = g^b mod p. They exchange A and B over the public network.

**Step 4 – Shared secret:** Alice computes K = B^a mod p and Bob computes K = A^b mod p. Both results equal g^(ab) mod p, so both now hold the same secret key K, which is then used as a symmetric session key (for example with AES).

**Numerical example:** Let p = 23 and g = 5. Alice chooses a = 6 and Bob chooses b = 15.

- Alice sends A = 5^6 mod 23 = 8.
- Bob sends B = 5^15 mod 23 = 19.
- Alice computes K = 19^6 mod 23 = 2.
- Bob computes K = 8^15 mod 23 = 2.

**Message flow summary:**

| Step | Alice | Public network | Bob |
|---|---|---|---|
| 1 | Knows p = 23, g = 5 | p and g are visible to everyone | Knows p = 23, g = 5 |
| 2 | Secret a = 6 | Nothing sent | Secret b = 15 |
| 3 | A = 5^6 mod 23 = 8 | A = 8 and B = 19 are visible | B = 5^15 mod 23 = 19 |
| 4 | K = 19^6 mod 23 = 2 | Nothing sent | K = 8^15 mod 23 = 2 |

The table shows that only p, g, A and B ever travel over the network. The private values a and b and the final key K stay on the end devices.

**Why the exchange is useful here:** the two users have no pre-shared secret and no secure channel, which is exactly the situation on a public network. DH solves the key-distribution problem and the resulting key K can then be fed into a key derivation function (KDF) to produce the actual encryption and MAC keys for the session.

Both sides obtain K = 2. An eavesdropper sees only p, g, A and B; without a or b, they cannot easily compute K. In practice p is at least 2048 bits long.

**(b) Identify a possible security threat and suggest a solution to mitigate it.**

**Threat – Man-in-the-Middle (MITM) attack:** Plain Diffie-Hellman provides no authentication. An active attacker, Mallory, sits between Alice and Bob. She intercepts A and sends Bob her own value M1, and intercepts B and sends Alice her own value

M2. As a result, Alice shares one key with Mallory and Bob shares a different key with Mallory. Mallory can decrypt, read, modify and re-encrypt all the traffic while both users believe they are communicating securely with each other.

**Solution – Authenticate the exchanged values:**

- **Digital signatures and certificates:** each party signs its DH public value with its private key (RSA or ECDSA) and the other party verifies the signature using a certificate issued by a trusted Certificate Authority (PKI / X.509).
- **Authenticated protocols:** use TLS (DHE/ECDHE) or IPsec/IKE, which combine ephemeral DH with authentication, and also give forward secrecy because a new key is used for every session.
- **Key confirmation with a MAC:** after deriving K, both sides exchange a MAC (such as HMAC) over the handshake to confirm they hold the same key.
- **Strong parameters:** use a large safe prime (2048 bits or more) or elliptic-curve DH (ECDH) to resist other attacks such as small-subgroup and precomputation attacks.

**How the MITM attack proceeds:**

| Step | Action |
|---|---|
| 1 | Alice sends A = g^a mod p towards Bob; Mallory intercepts it. |
| 2 | Mallory picks her own secret m and sends M = g^m mod p to Bob, pretending to be Alice. |
| 3 | Bob sends B = g^b mod p towards Alice; Mallory intercepts it and sends M to Alice, pretending to be Bob. |
| 4 | Alice computes K1 = M^a mod p (shared with Mallory); Bob computes K2 = M^b mod p (shared with Mallory). |
| 5 | Mallory decrypts with K1, reads or alters the data, and re-encrypts it with K2. Neither user notices. |

**Other threats to a DH exchange and their mitigation:**

| Threat | Description | Mitigation |
|---|---|---|
| Small-subgroup attack | Attacker sends a value from a tiny subgroup so the key falls in a small, guessable set | Validate received public values; use safe primes |
| Weak / short primes (Logjam) | Primes of 512 or 1024 bits can be broken by precomputation | Use p of 2048 bits or more, or ECDH |
| Replay attack | Old handshake messages are re-sent to trick a party | Use fresh ephemeral keys, nonces and timestamps |
| Key compromise | Long-term key stolen and past sessions decrypted | Use ephemeral DH (DHE/ECDHE) for forward secrecy |

With authenticated Diffie-Hellman, the users get a shared key that only they know and, in addition, confidence about who is on the other end.

**5. Cryptography for IoT Devices**

**(a) Analyze the constraints of IoT devices.**

Smart-home devices such as sensors, smart bulbs, door locks and thermostats are small and cheap, which places the following limits on the security design:

- **Limited processing power:** they use low-end 8/16/32-bit microcontrollers running at a few MHz, so heavy operations such as big-number modular exponentiation are slow.
- **Limited memory and storage:** only a few KB of RAM and limited flash, so large keys, certificates and big crypto libraries do not fit.
- **Low power / battery life:** many devices run for years on a coin cell or small battery, so each expensive computation and each transmitted byte drains energy.
- **Low bandwidth and small packets:** protocols such as Zigbee, BLE, LoRa and 6LoWPAN have small frames and low data rates, so large keys and signatures cause fragmentation and delays.
- **Real-time and latency needs:** a smart lock or alarm must respond quickly, so the handshake time must be short.
- **Physical exposure and scale:** devices are deployed in the open and can be tampered with, and a home may contain dozens of them, so key management must be simple and lightweight.

**Summary of constraints and their effect on the choice of algorithm:**

| Constraint | Typical limit | Effect on cryptography |
|---|---|---|
| Processor | 8 to 32-bit MCU, a few MHz to ~100 MHz | Avoid large modular exponentiation |
| Memory | 2 to 64 KB RAM, 32 to 512 KB flash | Need small keys and a small code size |
| Energy | Coin cell or battery for years | Fewer computations and transmitted bytes |
| Network | Zigbee / BLE / LoRa, 20 to 127 byte frames | Short keys and signatures avoid fragmentation |
| Security exposure | Physically accessible, rarely updated | Secure key storage and simple key management |

**(b) Recommend a suitable cryptographic approach and justify your answer.**

**Recommendation: Use Elliptic Curve Cryptography (ECC)** – ECDH (for example Curve25519 / X25519 or NIST P-256) for key exchange and ECDSA/EdDSA for authentication – and then use a lightweight symmetric cipher such as AES-128-GCM or ChaCha20-Poly1305 to encrypt the actual data (hybrid approach).

**Justification:** ECC gives the same security as RSA with much smaller keys, as shown below.

| Security level (bits) | RSA key size | ECC key size |
|---|---|---|
| 80 | 1024 bits | 160 bits |
| 112 | 2048 bits | 224 bits |

| Security level (bits) | RSA key size | ECC key size |
|---|---|---|
| 128 | 3072 bits | 256 bits |
| 192 | 7680 bits | 384 bits |
| 256 | 15360 bits | 521 bits |

- **Smaller keys and signatures:** a 256-bit ECC key is as strong as a 3072-bit RSA key, so less memory, storage and bandwidth is needed, which suits small packets and tiny devices.
- **Faster private-key operations:** ECC key generation, signing and key agreement are much faster than RSA at the same security level, which reduces handshake time.
- **Lower energy consumption:** fewer computations and fewer transmitted bytes extend battery life.
- **Scalability:** smaller certificates and keys make it practical to provision many devices in a home or city-wide network.
- **Strong security:** ECC relies on the elliptic curve discrete logarithm problem, which has no known sub-exponential attack, so well-chosen curves remain secure.

**RSA vs ECC for IoT at a glance:**

| Parameter | RSA (3072-bit) | ECC (256-bit) |
|---|---|---|
| Key size | 3072 bits | 256 bits |
| Signature size | About 384 bytes | About 64 bytes |
| Key generation | Slow (finding large primes) | Very fast |
| Private-key operation | Slow, high energy | Fast, low energy |
| Certificate / handshake size | Large | Small |
| Suitability for IoT | Poor | Very good |

**How ECDH works on a device:** both parties agree on a curve with a base point G. The device picks a private number d1 and computes the public point Q1 = d1 x G; the hub does the same with d2 and Q2 = d2 x G. After exchanging Q1 and Q2, each side computes the shared point d1 x Q2 = d2 x Q1, and the x-coordinate becomes the shared secret. Recovering d from Q is the elliptic curve discrete logarithm problem, which is infeasible for 256-bit curves.

**Real-world adoption:** modern IoT and smart-home standards such as Matter and TLS 1.3 / DTLS profiles for constrained devices rely on elliptic-curve cryptography (for example P-256 and Curve25519), which confirms that ECC is the practical choice for this class of devices.

**Limitations to be aware of:** ECC (like RSA) can be broken by a large quantum computer, so long-lived devices should be designed to allow firmware updates and a later move to post-quantum algorithms. ECC implementations must also protect against side-channel attacks and nonce reuse in ECDSA.

RSA is not ideal here because 2048–3072-bit keys, slow private-key operations and

large certificates are too heavy for low-power devices. Its only advantage is fast public-key (verification/encryption) operations and wide legacy support, which does not outweigh these costs.

**Implementation notes:** use a good hardware random number generator (weak randomness breaks ECDSA), store private keys in a secure element or protected memory, and use DTLS/TLS 1.3 or a similar standard protocol with ECDHE so that session keys are fresh for every connection.

**Conclusion:** ECC combined with symmetric encryption is the most suitable approach for secure, low-power IoT communication.